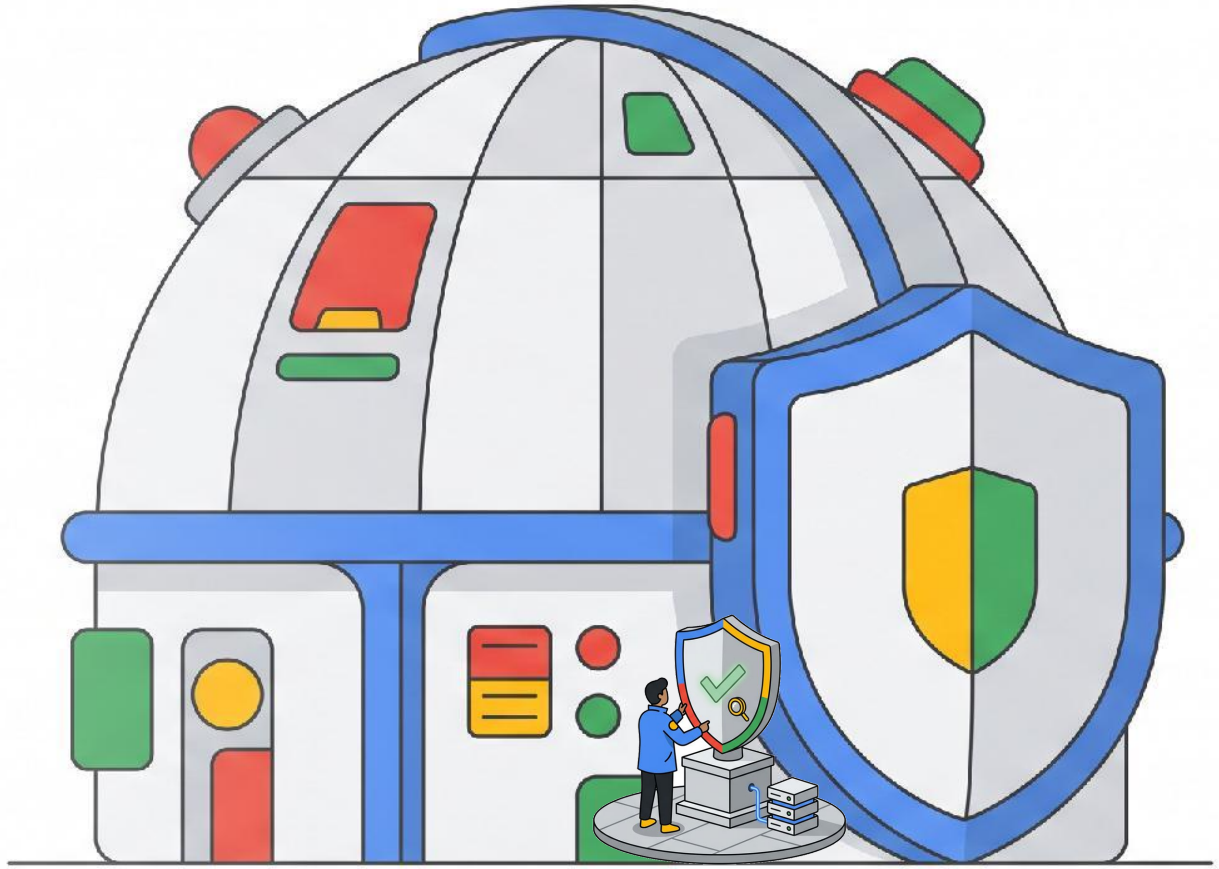
SECURITY BENCHMARK

# CodeMender deep-scan blueprint

From 4.7% baseline recall to 100% GT coverage in Python, Node.js & Java (Benchmark)



EVALUATION METRICS

# Deep scan lifts recall from 4.7% to 100%

**100%**

**Python GT recall**

Up from 4.7% R1; 0% FP (Benchmark)

**77.8%**

**Werkzeug 2.2.0 CVE**

7/9 mechanisms in R2 (Benchmark)

**118**

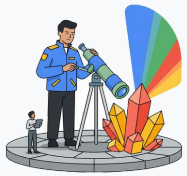
**Node.js Juice Shop**

4.92x lift, 19 Crit / 58 High (Benchmark)

**90**

**Java Spring Mall**

9.0x lift, 8 Crit / 36 High (Benchmark)



### Resolving 4.7% Round 1 recall

Round 1 quick scan (3.5 Flash, 60.5M tokens) hit 18/385 (4.7%) SQLi flaws; --deep unlocks full CWE coverage.



### 77.8%–100% Python recall

Round 2 hit 7/9 (77.8%) on Werkzeug 2.2.0 and 11/11 (100%, 0% FP) with 3.8 Flash Cyber & 4 Argon.



### Multi-language expansion

Validated across Python, Node.js (118 bugs), and Java (90 bugs), ready for C/C++, Ruby, and PHP apps.

Key evaluation takeaway: Pair Gemini 3.8 Flash Cyber or 4 Argon with --deep and semantic -c context.

TIMEOUT BOTTLENECK

# Overcome Round 1 4.7% recall bottlenecks

ROUND 1 BOTTLENECK: 4.7% RECALL

## Shallow scan & config gaps



- 3.5 Flash quick scan hit 18/385 (4.7%) across 14 apps (SQLi only)
- Default config.yaml omits .xml, .yml, .json, .pug (500 KB cap)
- Literal requirements.txt in --deep -c yields 0 bugs (py_13 trap)

ROUND 2 & 3 FIX: PARALLEL --DEEP

## 16 workers + semantic context

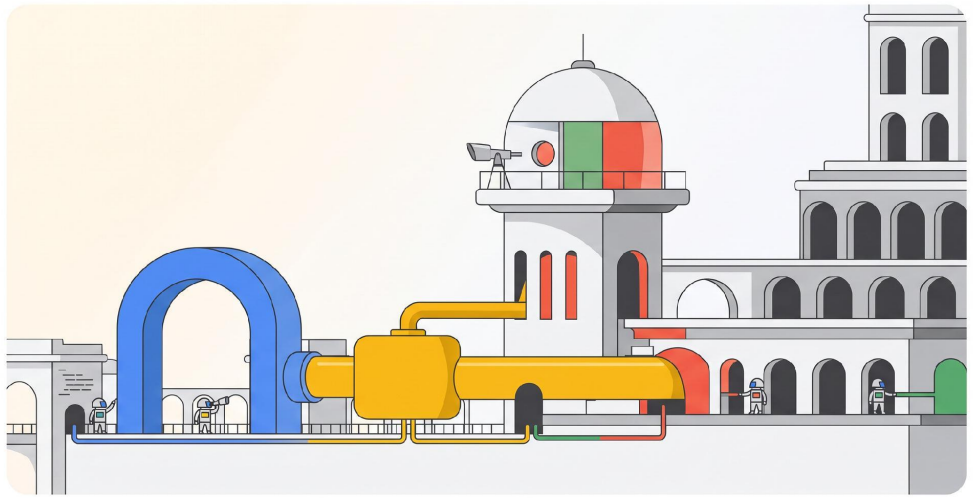


- --deep --deep-workers 16 --deep-max-turns 8 traverses all modules
- Semantic -c context (0 literal file.ext) hits 100% Python GT recall
- Unlocks 77.8% Werkzeug recall, 118 JS bugs, and 90 Java bugs

Critical v0.14.0 rule (py_13): Avoid literal file.ext in --deep -c so Tier 0 lockfile pruning never drops app code.

TAINT PIPELINE

# Werkzeug 2.2.0 hits 77.8% & fixes wrappers





### Werkzeug 2.2.0 deep evaluation

Recalled 7/12 entries (7/9 distinct mechanisms, 77.8%) at 85%–95% confidence after grouping #9–#12.

### 1. Recalled 7/9 distinct mechanisms (77.8% effective recall)

Caught multipart DoS (CVE-2023-25577/46136/49767), PIN RCE (CVE-2024-34069), and Request Smuggling.

### 2. Fix for #8 Thin-Wrapper Indirection (CVE-2023-23934)

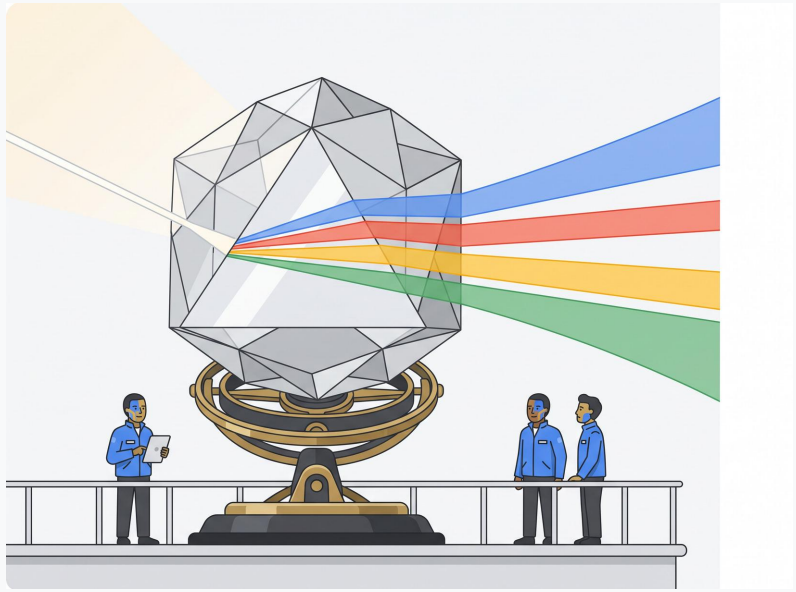
http.py (822k tokens) wraps sansio/http.py; co-locate caller + parser with PROJECT_FILE_INDEX.md.

### 3. Fix for #9–#12 Guard-Bypass Blind Spot (safe_join CVEs)

security.py (273k tokens) judged safe_join safe; add Windows device/UNC bypass rules to -c.

REASONING ENGINE

# Cyber & Argon models achieve 100% recall





## Priority model validation

3.8 Flash Cyber reaches 100% Python recall 45% faster (133s); 4 Argon leads Java (90 bugs) & deep Py (76s).

| Model & Mode | Target benchmark | Bugs | Crit / High | Recall / CWE | Sec |
|---|---|---|---|---|---|
| 3.5 Flash (Quick c0) | Python WSGI (2f) | 8 | 2 / 4 | 72.7% (8/11) | 126 |
| 3.8 Cyber (Quick c3) | Python WSGI (2f) | 11 | 3 / 4 | 100% (11/11) | 133 |
| 3.8 Flash (Quick c3) | Python WSGI (2f) | 13 | 4 / 6 | 100% (11/11) | 231 |
| 4 Argon (--deep c3) | Python WSGI (2f) | 11 | 4 / 4 | 100% (11/11) | 76 |
| 4 Argon (Quick c0) | Python WSGI (2f) | 10 | 3 / 5 | 90.9% (10/11) | 307 |
| 3.8 Flash (--deep c3) | Node.js bench (16f) | 118 | 19 / 58 | 35 CWEs | 224 |
| 4 Argon (--deep c3) | Java bench (20f) | 90 | 8 / 36 | 29 CWEs | 327 |
| 3.8 Flash (--deep c3) | Java bench (20f) | 80 | 7 / 32 | 29 CWEs | 253 |

Priority model ask validated: 3.8 Flash Cyber (133s) and 4 Argon (76s) both hit 100% (11/11) with 0% FP.

CONTEXT BLUEPRINT

# Hybrid -c context unlocks 100% GT recall



### Zero context baseline

**72.7% Py / 24 JS / 10 Java**

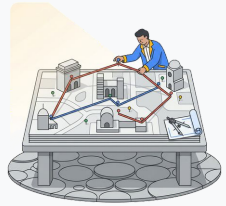
No -c flag; finds 8/11 app flaws (SQLi/RCE) but misses Werkzeug CVEs, PIN RCE & Cookie.



### SCA dependency manifest

**100% Py / 25 JS / 13 Java**

Supplies semantic versions (Werkzeug 2.2.0, jsonwebtoken, MyBatis) for CVE-2023-25577.



### SAST framework topology

**90.9% Py / 35 JS / 29 Java**

Maps WSGI handlers, Express routes, Spring controllers, and thin wrapper call chains.



### Hybrid max context brief

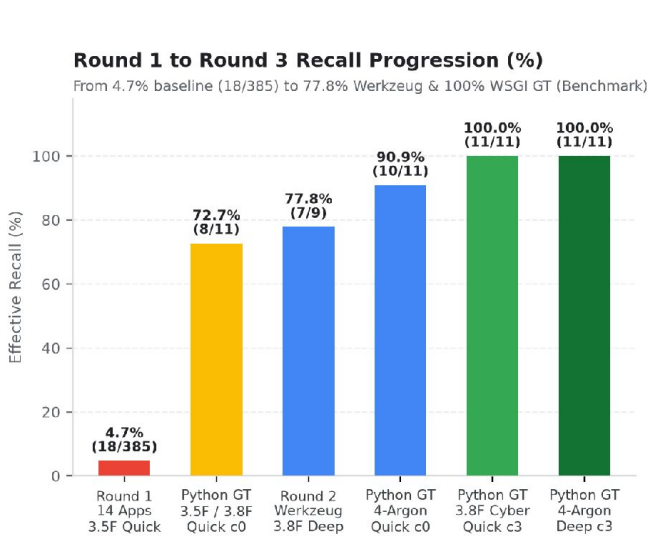
**100% Py / 118 JS / 90 Java**

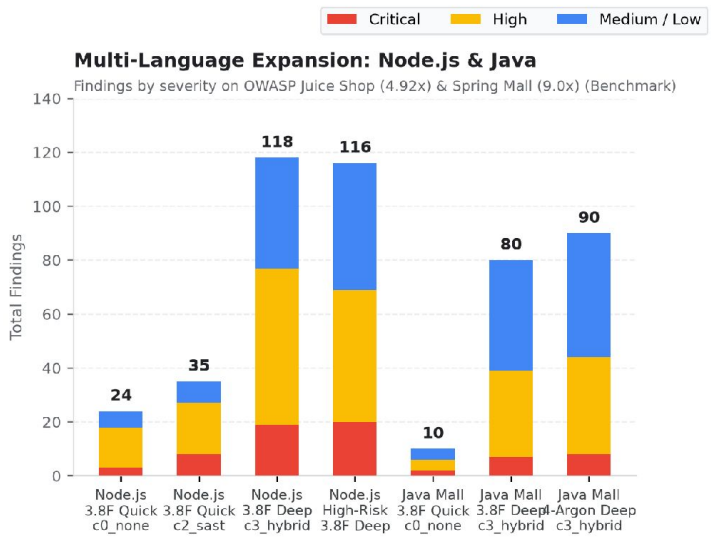
Combines semantic SCA, SAST sinks, and guard-bypass rules with 0% FP on safe_join.

Context blueprint rule: Inject semantic package versions & wrapper call chains without literal file.ext names.

ABLATION GROWTH

# Model, mode & context compound



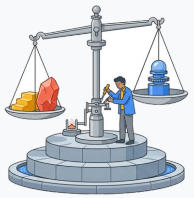




End-to-end progression: 4.7% R1 -> 77.8% Werkzeug R2 -> 100% Py GT, 118 JS & 90 Java bugs.



## Compounding recall gains

Upgrading from Round 1 quick scan to --deep + c3_hybrid_max lifts yield 4.92x in Node.js and 9.0x in Java.

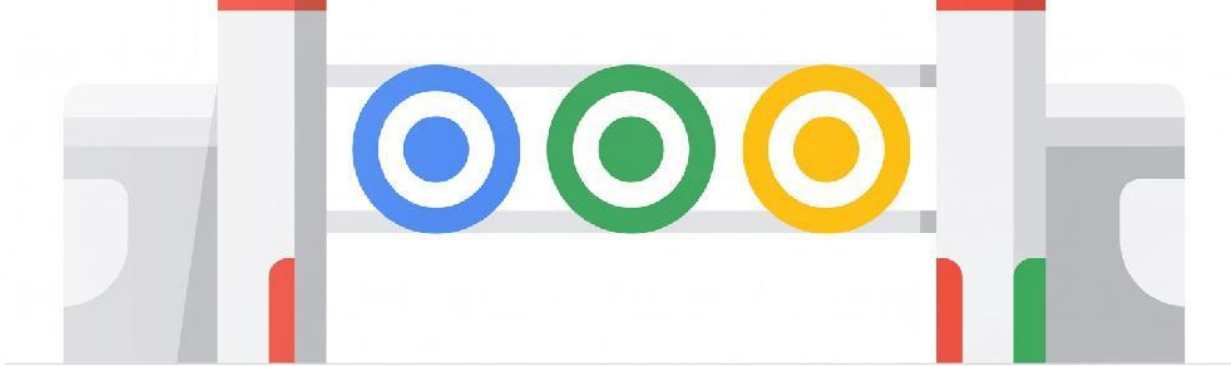


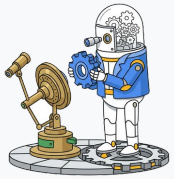
## 0% false-positive control

Verified by poc_exploit.py: 11/11 vulnerable endpoints detected with 0 false alarms on safe_join.

DIRECTORY SCOPING

# Tiered scoping scales across all 14 apps





### C/C++ & Python modules

**1–10 files · 1–3 min**

Scan crypto/protocol libs and Python WSGI apps with Quick+c3 or Argon --deep (100% GT).



### Web backends (JS/Java/Ruby)

**15–50 files · 3–6 min**

Pair 16-worker --deep with PROJECT_FILE_INDEX.md for 118 JS & 90 Java bugs.



### Ops, CI & key-mgmt repos

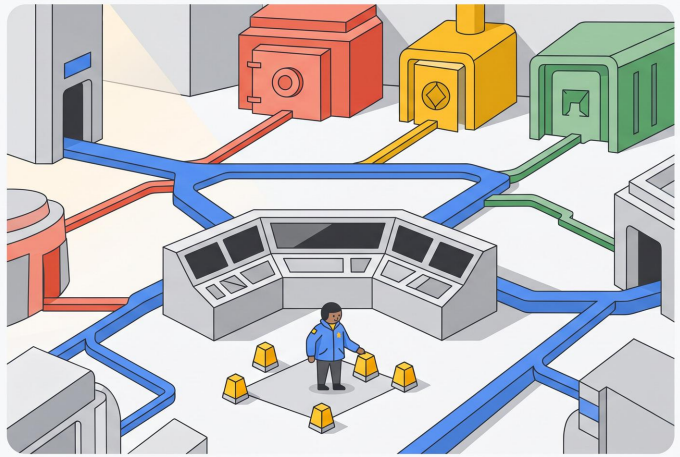
**120–300+ files · 10–15 min**

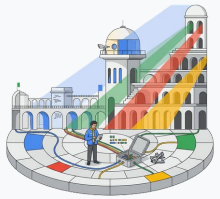
Partition monoliths by service boundary and exclude generated ORM POJOs.

| 14-App target category | Files | Recommended mode & model | Verified benchmark yield | Time | Key scoping rule |
|---|---|---|---|---|---|
| Python WSGI / C-C++ libs | 2–15 | 3.8 Cyber Quick or 4 Argon --deep | 100% (11/11) GT / 77.8% CVE | 76s–133s | Co-locate wrappers (.h/.c, sansio) |
| Node.js / TS web apps | 16–35 | 3.8 Flash (--deep w=16 t=8 + c3) | 118 bugs (19 Crit / 58 High) | 224s–536s | Include routes/.ts, lib/, views/.pug |
| Java / Spring backends | 20–48 | 4 Argon (--deep w=16 t=8 + c3) | 90 bugs (8 Crit / 36 High) | 327s–672s | Add .xml/.yml; exclude mall-mbg |
| Ops / CI / Ruby / PHP | 30–100 | 3.8 Flash / 4 Argon (--deep + c3) | Full multi-CWE traversal | 5–12m | Partition auth, key-mgmt & CI |

TYPESCRIPT ROUTES

# Node.js deep scan captures 118 findings





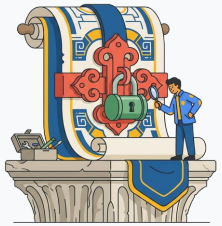
### 4.92x over quick baseline

Lifts Node.js yield from 24 (Quick c0) to 118 bugs (19 Crit, 58 High, 35 CWEs) in 223.7s.

| Vulnerability class (CWE) | Bugs | Representative files & sinks (Benchmark) |
|---|---|---|
| CWE-94 / 1336 Code & SSTI RCE | 14 | views/promotionVideo.pug, b2bOrder.ts (vm) |
| CWE-89 SQL Injection | 8 | routes/search.ts, login.ts (Sequelize raw) |
| CWE-943 NoSQL Injection | 9 | routes/showProductReviews.ts, trackOrder.ts |
| CWE-918 Server-Side Request Forgery | 5 | routes/profileImageUrlUpload.ts (HTTP fetch) |
| CWE-22 / 611 Path Traversal & XXE | 12 | routes/fileUpload.ts (Zip Slip & libxmljs XXE) |
| CWE-347 / 327 JWT & Crypto Flaws | 15 | lib/insecurity.ts (RS256->HS256, MD5, keys) |
| CWE-639 / 862 Broken Access Control | 21 | routes/basket.ts (IDOR), dataErasure.ts |
| CWE-79 / 601 XSS & Open Redirect | 16 | routes/redirect.ts, userProfile.ts (XSS) |

MYBATIS MAPPERS

# Spring Boot & MyBatis rules find 90 bugs



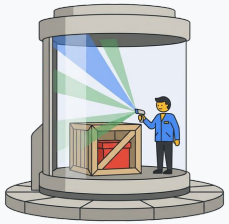
### MyBatis XML SQL injection

Adding .xml to config.yaml exposes ${} interpolation in OmsOrderDao.xml and PmsProductDao.xml (8 Critical).



### Broken auth & order IDOR

Pairing portal controllers with SecurityConfig exposes unauthenticated admin routes and order IDOR (36 High).



### YAML secrets & OSS upload

Adding .yml to config.yaml catches hardcoded JWT/DB secrets in application.yml and MinioController XSS.

Why default Java scans miss bugs: Round 1 config.yaml omitted .xml and .yml—adding them lifts Mall 9.0x (10 -> 90).

PIPELINE AUTOMATION

# Upgrade config.yaml & CLI for all 14 apps

## Required upgrades over Round 1 config.yaml & CLI

### 1. Expand ~/.codemender/config.yaml & set incremental: false

include: [.py,.java,.go,.js,.ts,.c,.cc,.cpp,.h,.rb,.php,.xml,.yml,.json,.pug,.txt]

### 2. Inject semantic c3_hybrid_max (-c with 0 literal file.ext)

export CM_CTX="Read file index; check CVE versions, wrappers & safe_join bypass."

### 3. Purge cache (cm clean) & run 16-worker --deep (v0.14.0)

cm clean && cm -y --sandbox=false find . --deep --deep-workers 16 --deep-max-turns 8 -c "$CM_CTX"



### Fix Round 1 config.yaml

Round 1 config.yaml had incremental: true and 500 KB cap; set incremental: false and 1000 KB for full audits.

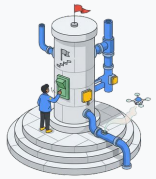


### SAST baseline alignment

Run cm clean before each benchmark and export cm report -f json to compare against existing SAST tools.

ROLLOUT ROADMAP

# Roadmap from >75% recall to cm fix



### Phase 1: Model & config upgrade

Enable 3.8 Flash Cyber & 4 Argon; deploy expanded config.yaml and cm v0.14.0.



### Phase 2: 14-app SAST benchmark

Re-scan 385 historical bugs in C/C++, Java, JS & Python with --deep + c3.



### Phase 3: Verify & cm fix rollout

Once recall exceeds 75%–90%, expand from pure detection to cm verify & cm fix.

| Evaluation requirement | Round 1 / 2 bottleneck | Recommended v0.14.0 setup | Verified benchmark outcome |
|---|---|---|---|
| 1. Overcome 4.7% R1 recall | 18/385 (4.7%) on 3.5 Flash quick | --deep (w=16, t=8) + Cyber / Argon | 77.8% Werkzeug & 100% Py GT |
| 2. Wrapper & guard gaps | Missed #8 sansio & #9–12 safe_join | Co-locate slices + guard-bypass -c | 100% recall, 0% FP on safe_join |
| 3. Multi-language scale | 14 apps (C/C++, Java, JS, Py, Ruby) | Add .xml/.yml/.pug + FILE_INDEX | 118 JS (4.9x) & 90 Java (9.0x) |
| 4. Automated patching | Detection focus until recall >75% | Gate cm verify & cm fix on CWEs | Ready for Phase 3 patch trials |